

Debt Collection Call Analysis – Technical Report

GitHub Link: [call-guard](#)

1. Introduction

As part of this project, we analyzed conversations between debt collection agents and borrowers to evaluate compliance, professionalism, and call metrics. The dataset consists of JSON files where each file represents a call, containing detailed utterance-level information such as the speaker, speech content, and timestamps.

This report documents the methodologies used, implementation details, and findings related to:

- **Profanity Detection**
- **Privacy and Compliance Violation Detection**
- **Call Quality Metrics Analysis**

2. Data Structure

Note: Provided Zip had JSON files instead of YAML files.

Each JSON file follows the structure below:

- **speaker:** Identifies whether the utterance is from an agent or borrower.
- **text:** Contains the actual speech content.
- **stime:** Start timestamp of the utterance.
- **etime:** End timestamp of the utterance.

3. Implementation Approach

3.1 Profanity Detection

Objective: Identify all calls where collection agents and borrowers used profane language.

Approach 1: Pattern Matching (Regex-Based Detection)

- Implemented a predefined list of profane words using regex pattern matching.
- Scanned conversations to detect flagged words and phrases.

Approach 2: Machine Learning (ML-Based Detection)

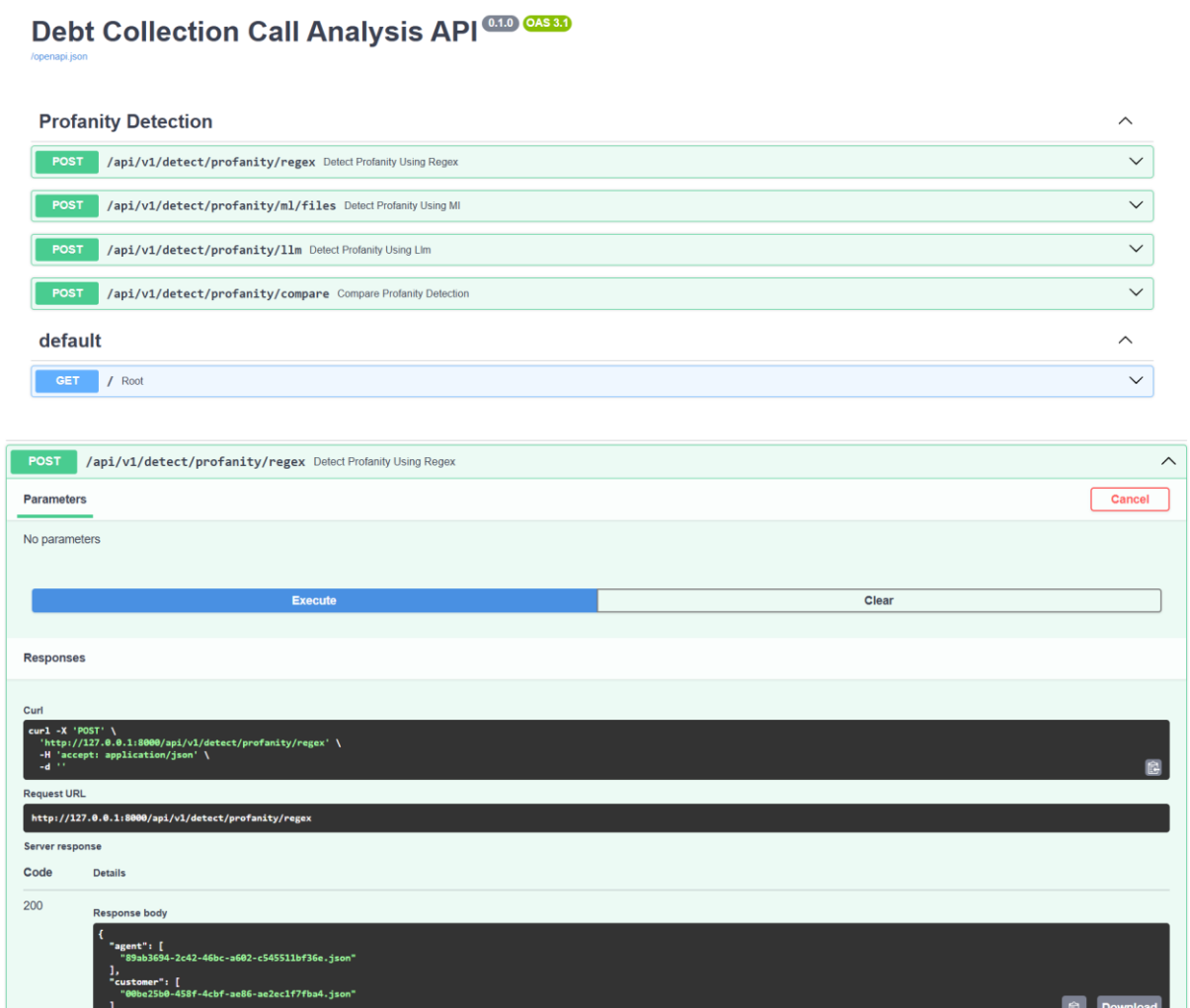
- Used a Hugging Face transformer model [toxic-bert] to classify utterances as profane or not.
- Model inferred whether an utterance contained offensive language.

Comparison & Findings

Approach	Advantages	Limitations
Regex-Based	Fast and interpretable	Limited to predefined words, cannot detect obfuscated profanity
ML-Based	Context-aware, can detect indirect profanity.	Requires a pre-trained model and API integration

Recommendation: The ML-based approach is more robust as it captures contextual profanity beyond a predefined word list.

Refer the images below for comparison.



3.2 Privacy and Compliance Violation Detection

Objective: Identify all calls where agents shared sensitive information (balance/account details) without verifying the borrower's identity (DOB, address, or SSN).

Approach 1: Pattern Matching (Regex-Based Detection)

- Defined keyword patterns for **account details**, **balance** inquiries, and **identity verification**.

- Detected instances where balance/account details were shared without prior identity verification.

Approach 2: Machine Learning (Hugging Face LLM for Classification)

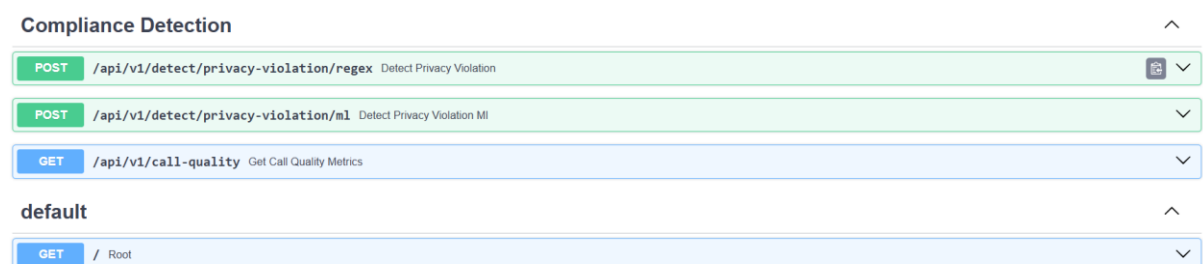
- Used a **zero-shot classification model** (BART-based) to determine if an utterance involved sensitive information sharing without verification.
- Classified utterances into “**financial information**”, “**account details**”, or “**identity verification**” categories.

Comparison & Findings

Approach	Advantages	Limitations
Regex-Based	Easy to implement, works well for structured phrases	Cannot handle paraphrased or complex statements
ML-Based	Can detect nuanced violations without explicit keyword presence	Requires API integration and computational resources

Recommendation: The ML-based approach is preferable as it captures implicit privacy violations better than regex-based rules.

Refer below images:



4. Call Quality Metrics Analysis

Objective: Calculate **overtalk percentage** and **silence percentage** per call and visualize them.

Implementation

- Parsed each JSON file and computed:
 - **Overtalk:** Duration where two speakers overlapped.
 - **Silence:** Gaps between consecutive utterances.
- Calculated percentages relative to the total call duration.

Visualization

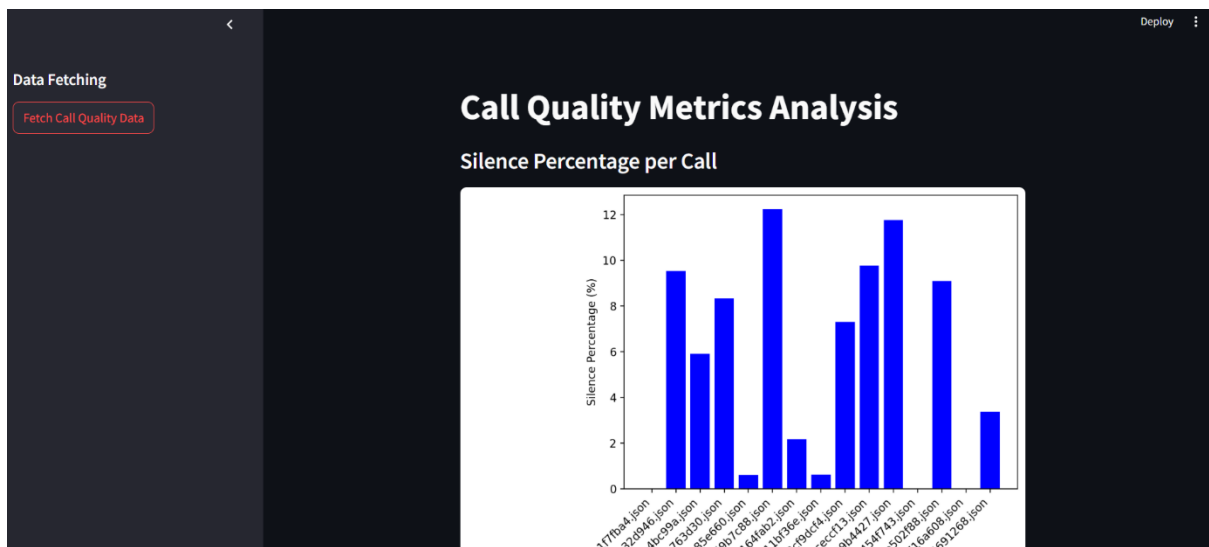
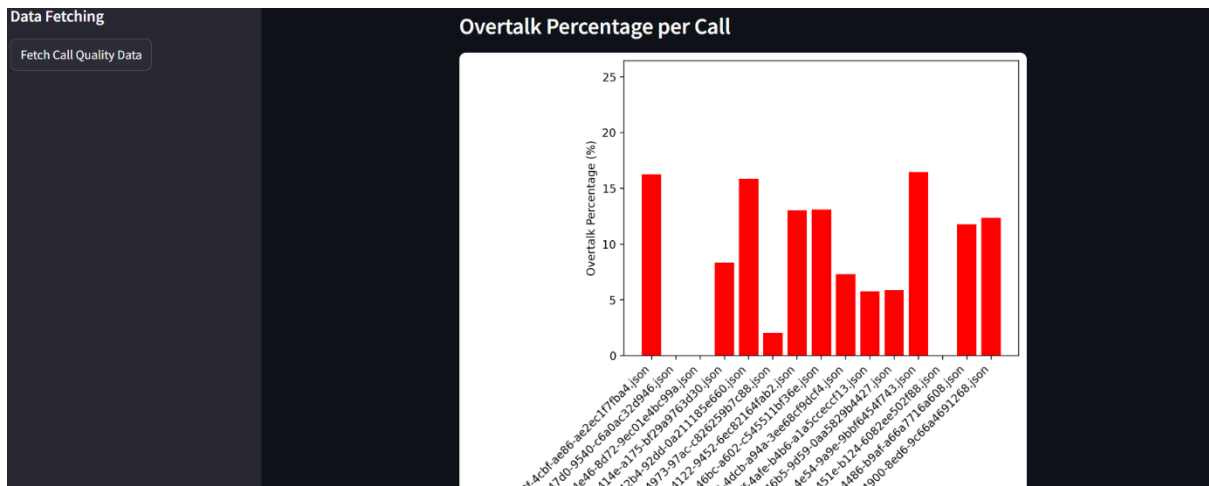
- **Streamlit Dashboard:**

- Displayed bar charts for overtalk and silence percentages.
- Allowed users to analyze trends in call quality.

Findings

- Some calls had overtalk exceeding **16%**, indicating frequent interruptions.
- Silence percentages varied significantly across different calls.

Refer below images:



5. GitHub Repository Details

The repository includes:

- **Backend:** FastAPI endpoints for profanity detection, privacy violation detection, and call quality analysis.
- **Frontend:** Streamlit dashboard for interactive analysis.

- **Documentation:** Setup and execution instructions in readMe and
-

6. Conclusion & Future Work

- **Profanity & Compliance Violations:** ML-based approaches were more effective than regex.
- **Call Quality Metrics:** The visualizations provided insights into agent-borrower interactions.

Future Enhancements:

- Train a custom profanity detection model for better accuracy.
 - Improve call quality analysis by integrating speaker diarization models.
-

7. References & Tools Used

- Github link: [call-guard](#)
- OpenApi: <https://chatgpt.com>
- Toxic Bert: <https://huggingface.co/unitary/toxic-bert>
- Bart Large MNLI: <https://huggingface.co/facebook/bart-large-mnli>
- Hugging Face: <https://huggingface.co/>
- <https://docs.streamlit.io/develop/api-reference/charts/st.pyplot>
- <https://discuss.streamlit.io/t/how-to-display-matplotlib-graphs-in-streamlit-application/35383>
- <https://www.freecodecamp.org/news/get-started-with-hugging-face/>
- <https://medium.com/data-science/choosing-and-implementing-hugging-face-models-026d71426fbe>
- <https://medium.com/@avinashmachinelearninginfo/hands-on-with-bert-model-c77a27b9839b>
- <https://www.youtube.com/watch?v=bFB4zqkcatU>
- <https://www.youtube.com/watch?v=GLO5FZzfrS0>